

Loops in Python

In the previous modules, we learned variables, strings, input, calculations, and decision making using `if`, `elif`, and `else`.

A loop allows a program to repeat a block of code.

For example, suppose we want to display the word:

```
1 Python
```

ten times.

Without a loop, we would need to write:

```
1 print("Python")
2 print("Python")
3 print("Python")
4 print("Python")
5 print("Python")
6 print("Python")
7 print("Python")
8 print("Python")
9 print("Python")
10 print("Python")
```

This is repetitive and inefficient.

Using a loop, we can tell Python to repeat the instruction automatically.

In this module, we will learn:

- What loops are
- The `while` loop
- Loop conditions
- Updating a loop variable
- Infinite loops
- The `for` loop
- The `range()` function
- Different ways to use `range()`
- Looping through strings

- Nested loops
- The `break` statement
- The `continue` statement
- Looping with conditions
- Practical programs using loops

What is a Loop?

A loop is a programming structure that repeats a block of code.

A loop is useful when:

- The same action must be performed many times.
- We want to process a sequence of values.
- We do not know in advance exactly how many times an action must be repeated.

Python provides two main types of loops:

- `while` loop
- `for` loop

The while Loop

A `while` loop repeats a block of code as long as a condition remains true.

Basic structure:

```
1 while condition:
2     statement
```

Example:

```
1 number = 1
2
3 while number <= 5:
4     print(number)
5     number = number + 1
```

Output:

```
1  1
2  2
3  3
4  4
5  5
```

How a while Loop Works

Consider:

```
1  number = 1
2
3  while number <= 5:
4      print(number)
5      number = number + 1
```

The process is:

1. `number` starts with the value 1.
2. Python checks whether `number <= 5`.
3. The condition is true, so the loop body runs.
4. Python prints 1.
5. `number` becomes 2.
6. The condition is checked again.
7. The process continues.
8. When `number` becomes 6, the condition is false.
9. The loop stops.

The important idea is that the condition is checked before each iteration.

Updating the Loop Variable

A common pattern with a `while` loop is:

```
1 counter = 1
2
3 while counter <= 5:
4     print(counter)
5     counter = counter + 1
```

The variable `counter` controls how long the loop continues.

If we forget to change the value of `counter`, the condition may remain true forever.

For example:

```
1 counter = 1
2
3 while counter <= 5:
4     print(counter)
```

This keeps printing 1 because `counter` never changes.

Such a loop is called an **infinite loop**.

Infinite Loops

An infinite loop is a loop that never reaches a condition that becomes false.

For example:

```
1 number = 1
2
3 while number <= 5:
4     print(number)
```

The program will continue printing 1.

When testing programs, an infinite loop can make the program appear to be stuck.

A correct loop should normally contain a way for its condition to eventually become false.

Counting Backward With `while`

A `while` loop can also count downward.

Example:

```
1 number = 5
2
3 while number >= 1:
4     print(number)
5     number = number - 1
```

Output:

```
1 5
2 4
3 3
4 2
5 1
```

Here the variable becomes smaller after every iteration.

Using while With User Input

A loop can repeatedly ask the user for information.

For example:

```
1 number = 1
2
3 while number <= 3:
4     name = input("Enter a name: ")
5     print("Hello", name)
6     number = number + 1
```

The program asks for three names.

The for Loop

The **for** loop is commonly used when we want to repeat an action for each item in a sequence or for a fixed number of times.

For example:

```
1 for i in range(5):
2     print(i)
```

Output:

```
1 0
2 1
3 2
4 3
5 4
```

The `for` loop has automatically handled the repetition.

The `range()` Function

The `range()` function is frequently used with `for` loops.

The basic form is:

```
1 range(stop)
```

For example:

```
1 range(5)
```

produces the sequence:

```
1 0, 1, 2, 3, 4
```

The stop value is not included.

Therefore:

```
1 for i in range(5):
2     print(i)
```

prints 0 through 4, not 5.

Using Start and Stop With `range()`

We can specify both a starting value and a stopping value.

Syntax:

```
1 range(start, stop)
```

Example:

```
1 for i in range(1, 6):
2     print(i)
```

Output:

```
1  1
2  2
3  3
4  4
5  5
```

The starting value is included.

The stopping value is excluded.

Therefore:

```
1  range(1, 6)
```

means:

```
1  1, 2, 3, 4, 5
```

Using a Step With range()

We can also specify how much the value should change after each iteration.

Syntax:

```
1  range(start, stop, step)
```

Example:

```
1  for i in range(1, 10, 2):
2      print(i)
```

Output:

```
1  1
2  3
3  5
4  7
5  9
```

The value increases by 2 after each iteration.

Another example:

```
1  for i in range(2, 11, 2):
2      print(i)
```

Output:

```
1  2
2  4
3  6
4  8
5 10
```

Counting Backward With range()

The step can also be negative.

Example:

```
1  for i in range(5, 0, -1):
2      print(i)
```

Output:

```
1  5
2  4
3  3
4  2
5  1
```

The stop value 0 is not included.

Looping Through a String

A for loop can also process the characters of a string one by one.

Example:

```
1  word = "Python"
2
3  for character in word:
4      print(character)
```

Output:

```
1  P
2  y
3  t
4  h
```

```
5 o
6 n
```

The variable `character` contains one character at a time.

Another example:

```
1 word = "Python"
2
3 for character in word:
4     print(character, end=" ")
```

Output:

```
1 P y t h o n
```

The `end` argument changes what `print()` places after each output.

Calculating a Sum Using a Loop

Loops can be used to perform repeated calculations.

For example, suppose we want to calculate the sum of numbers from 1 to 5.

```
1 total = 0
2
3 for i in range(1, 6):
4     total = total + i
5
6 print(total)
```

Output:

```
1 15
```

The calculation is:

$$1 + 2 + 3 + 4 + 5 = 15$$

The variable `total` is called an accumulator because it keeps the running total.

Multiplication Tables

Loops make it easy to generate multiplication tables.

Example:

```
1 number = int(input("Enter a number: "))
2
3 for i in range(1, 11):
4     print(number, "x", i, "=", number * i)
```

If the user enters 5, the output is:

```
1 5 x 1 = 5
2 5 x 2 = 10
3 5 x 3 = 15
4 5 x 4 = 20
5 5 x 5 = 25
6 5 x 6 = 30
7 5 x 7 = 35
8 5 x 8 = 40
9 5 x 9 = 45
10 5 x 10 = 50
```

Nested Loops

A loop can be placed inside another loop.

This is called a **nested loop**.

Example:

```
1 for i in range(1, 4):
2     for j in range(1, 4):
3         print(i, j)
```

Output:

```
1 1 1
2 1 2
3 1 3
4 2 1
5 2 2
6 2 3
7 3 1
8 3 2
9 3 3
```

The inner loop runs completely for every iteration of the outer loop.

Creating Patterns With Loops

Loops can be used to create simple patterns.

Example:

```
1  for i in range(1, 6):  
2      print("*" * i)
```

Output:

```
1  *  
2  **  
3  ***  
4  ****  
5  *****
```

The number of asterisks increases by one on each line.

Another example:

```
1  for i in range(5, 0, -1):  
2      print("*" * i)
```

Output:

```
1  *****  
2  *****  
3  ***  
4  **  
5  *
```

The break Statement

The **break** statement immediately stops a loop.

Example:

```
1  for i in range(1, 11):  
2      if i == 6:  
3          break  
4      print(i)
```

Output:

```
1  1
2  2
3  3
4  4
5  5
```

When `i` becomes 6, the `break` statement stops the loop.
The remaining values are not processed.

The `continue` Statement

The `continue` statement skips the current iteration and moves to the next iteration.

Example:

```
1  for i in range(1, 6):
2      if i == 3:
3          continue
4      print(i)
```

Output:

```
1  1
2  2
3  4
4  5
```

When `i` is 3, Python skips the `print()` statement for that iteration.

Using Conditions Inside Loops

We can place an `if` statement inside a loop.

Example:

```
1  for i in range(1, 11):
2      if i % 2 == 0:
3          print(i)
```

Output:

```
1  2
2  4
```

```
3 6
4 8
5 10
```

The loop examines every number, but only the even numbers are printed.

Printing Odd Numbers

We can similarly print odd numbers.

```
1 for i in range(1, 11):
2     if i % 2 != 0:
3         print(i)
```

Output:

```
1 1
2 3
3 5
4 7
5 9
```

Counting Values With a Loop

An accumulator can also be used to count how many times something happens.

Example:

```
1 count = 0
2
3 for i in range(1, 11):
4     if i % 2 == 0:
5         count = count + 1
6
7 print("Even numbers:", count)
```

Output:

```
1 Even numbers: 5
```

The variable `count` increases whenever an even number is found.

while Versus for

Both loops repeat code, but they are commonly used in different situations.

Use a **for** loop when you are working through a sequence or when the number of repetitions is known.

Example:

```
1  for i in range(1, 11):  
2      print(i)
```

Use a **while** loop when repetition depends on a condition.

Example:

```
1  password = ""  
2  
3  while password != "python123":  
4      password = input("Enter password: ")  
5  
6  print("Correct password.")
```

The **while** loop continues until the required condition is satisfied.

Practice Exercises

Attempt each exercise yourself before looking at the answers.

Exercise 1: Print Numbers

Use a **for** loop to print the numbers from 1 to 10.

Exercise 2: Print Even Numbers

Use a **for** loop to print all even numbers from 1 to 20.

Exercise 3: Print Odd Numbers

Use a **for** loop to print all odd numbers from 1 to 20.

Exercise 4: Countdown

Use a **for** loop to print:

1	10
2	9
3	8
4	7
5	6
6	5
7	4
8	3
9	2
10	1

Exercise 5: Multiplication Table

Ask the user to enter a number.

Print its multiplication table from 1 to 10.

Exercise 6: Sum From 1 to 100

Use a loop to calculate the sum of all numbers from 1 to 100.

Exercise 7: Sum of Even Numbers

Calculate the sum of all even numbers from 1 to 100.

Exercise 8: Count Even Numbers

Count how many even numbers exist between 1 and 100.

Exercise 9: Count Characters

Ask the user to enter a word.

Use a loop to print each character on a separate line.

Exercise 10: Count Vowels

Ask the user to enter a word or sentence.

Use a loop to count the number of vowels.

Count:

```
1  a
2  e
3  i
4  o
5  u
```

Make the program work for both uppercase and lowercase letters.

Exercise 11: Number Divisibility

Print all numbers from 1 to 50 that are divisible by both 3 and 5.

Exercise 12: Square of Numbers

Print the square of every number from 1 to 10.

Expected format:

```
1  1 1
2  2 4
3  3 9
4  ...
5 10 100
```

Exercise 13: Factorial

Ask the user to enter a positive integer.

Calculate its factorial using a loop.

For example:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Exercise 14: Reverse Countdown With While

Use a `while` loop to print numbers from 10 down to 1.

Exercise 15: Password Loop

Ask the user to enter a password repeatedly.

Continue asking until the user enters:

```
1 python123
```

Then display:

```
1 Access Granted
```

Exercise 16: Stop at Five

Use a loop from 1 to 10.

Use **break** to stop the loop when the value reaches 5.

Exercise 17: Skip Multiples of Three

Use a loop from 1 to 20.

Use **continue** to skip numbers divisible by 3.

Exercise 18: Star Pattern

Use a loop to produce:

```
1 *
2 **
3 ***
4 ****
5 *****
```

Exercise 19: Reverse Star Pattern

Use a loop to produce:

```
1 *****
2 ****
3 ***
4 **
5 *
```

Exercise 20: Number Triangle

Use nested loops to produce:

```
1  1
2  12
3  123
4  1234
5  12345
```

Answers to Practice Exercises

Answer 1: Print Numbers

```
1  for i in range(1, 11):
2      print(i)
```

Answer 2: Print Even Numbers

```
1  for i in range(1, 21):
2      if i % 2 == 0:
3          print(i)
```

Answer 3: Print Odd Numbers

```
1  for i in range(1, 21):
2      if i % 2 != 0:
3          print(i)
```

Answer 4: Countdown

```
1  for i in range(10, 0, -1):
2      print(i)
```

Answer 5: Multiplication Table

```
1 number = int(input("Enter a number: "))
2
3 for i in range(1, 11):
4     print(number, "x", i, "=", number * i)
```

Answer 6: Sum From 1 to 100

```
1 total = 0
2
3 for i in range(1, 101):
4     total = total + i
5
6 print(total)
```

Output:

```
1 5050
```

Answer 7: Sum of Even Numbers

```
1 total = 0
2
3 for i in range(1, 101):
4     if i % 2 == 0:
5         total = total + i
6
7 print(total)
```

Output:

```
1 2550
```

Answer 8: Count Even Numbers

```
1 count = 0
2
3 for i in range(1, 101):
4     if i % 2 == 0:
```

```
5         count = count + 1
6
7     print(count)
```

Output:

```
1     50
```

Answer 9: Count Characters

```
1 word = input("Enter a word: ")
2
3 for character in word:
4     print(character)
```

Answer 10: Count Vowels

```
1 text = input("Enter a word or sentence: ")
2
3 count = 0
4
5 for character in text.lower():
6     if character in "aeiou":
7         count = count + 1
8
9 print("Number of vowels:", count)
```

Answer 11: Number Divisibility

```
1 for i in range(1, 51):
2     if i % 3 == 0 and i % 5 == 0:
3         print(i)
```

Output:

```
1     15
2     30
3     45
```

Answer 12: Square of Numbers

```
1 for i in range(1, 11):  
2     print(i, i * i)
```

Output:

```
1 1 1  
2 2 4  
3 3 9  
4 4 16  
5 5 25  
6 6 36  
7 7 49  
8 8 64  
9 9 81  
10 10 100
```

Answer 13: Factorial

```
1 number = int(input("Enter a positive integer: "))  
2  
3 factorial = 1  
4  
5 for i in range(1, number + 1):  
6     factorial = factorial * i  
7  
8 print("Factorial:", factorial)
```

Answer 14: Reverse Countdown With While

```
1 number = 10  
2  
3 while number >= 1:  
4     print(number)  
5     number = number - 1
```

Answer 15: Password Loop

```
1 password = ""
2
3 while password != "python123":
4     password = input("Enter password: ")
5
6 print("Access Granted")
```

Answer 16: Stop at Five

```
1 for i in range(1, 11):
2     if i == 5:
3         break
4     print(i)
```

Output:

```
1 1
2 2
3 3
4 4
```

Answer 17: Skip Multiples of Three

```
1 for i in range(1, 21):
2     if i % 3 == 0:
3         continue
4     print(i)
```

Answer 18: Star Pattern

```
1 for i in range(1, 6):
2     print("*" * i)
```

Output:

```
1 *
2 **
```

```
3  ***
4  ****
5  *****
```

Answer 19: Reverse Star Pattern

```
1  for i in range(5, 0, -1):
2      print("*" * i)
```

Output:

```
1  *****
2  ****
3  ***
4  **
5  *
```

Answer 20: Number Triangle

```
1  for i in range(1, 6):
2      for j in range(1, i + 1):
3          print(j, end="")
4      print()
```

Output:

```
1  1
2  12
3  123
4  1234
5  12345
```

Mini Project: Student Marks Analyzer

Create a program that asks the user to enter the marks of 5 subjects.

The program should:

- Calculate the total marks.

- Calculate the average marks.
- Count how many subjects have marks of 40 or above.
- Count how many subjects have marks below 40.

The program should display the total, average, number of passed subjects, and number of failed subjects.

Example:

```
1  Enter marks for Subject 1: 75
2  Enter marks for Subject 2: 82
3  Enter marks for Subject 3: 68
4  Enter marks for Subject 4: 35
5  Enter marks for Subject 5: 91
6
7  Total: 351
8  Average: 70.2
9  Passed Subjects: 4
10 Failed Subjects: 1
```

Try to write the program using a loop instead of writing five separate input statements.

Answer: Student Marks Analyzer

```
1  total = 0
2  passed = 0
3  failed = 0
4
5  for i in range(1, 6):
6      marks = float(input(f"Enter marks for Subject {i}: "))
7
8      total = total + marks
9
10     if marks >= 40:
11         passed = passed + 1
12     else:
13         failed = failed + 1
14
15 average = total / 5
16
```

```
17 print()
18 print("Total:", total)
19 print("Average:", average)
20 print("Passed Subjects:", passed)
21 print("Failed Subjects:", failed)
```

Final Practice Task

Create a number guessing program.

Store a secret number:

```
1 secret = 7
```

The program should repeatedly ask the user to guess the number.

If the guess is smaller than the secret number, display:

```
1 Too low.
```

If the guess is greater than the secret number, display:

```
1 Too high.
```

If the guess is correct, display:

```
1 Correct!
```

The program must continue until the correct number is entered.

Answer: Number Guessing Program

```
1 secret = 7
2
3 guess = 0
4
5 while guess != secret:
6     guess = int(input("Enter your guess: "))
7
8     if guess < secret:
9         print("Too low.")
10    elif guess > secret:
11        print("Too high.")
12    else:
```

13

```
print("Correct!")
```

Module Review

Before moving to the next module, make sure you understand:

- What a loop is.
- How the **while** loop works.
- How to control a **while** loop with a condition.
- Why a loop variable must normally be updated.
- What an infinite loop is.
- How to count upward and downward.
- How the **for** loop works.
- How **range()** works.
- The difference between **range(stop)** and **range(start, stop)**.
- How the step value works.
- How to use a negative step.
- How to loop through a string.
- How to use loops for calculations.
- How to use **break**.
- How to use **continue**.
- How to place conditions inside loops.
- What nested loops are.
- How to create simple patterns using loops.
- How to use loops in practical programs.

End of Python Module 4